



Unique Identification Authority of India



UIDAI Data Hackathon 2026

UIDAI Data Hackathon 2026

Data-Driven Governance · Real-Time Analytics ·
Compliance Monitoring

Team Members:

1. Aditya Gupta
2. Harsh Singla
3. Kris Garg
4. Kavya Arora
5. Daksh Mittal



Problem Statement: Unlocking Societal Trends in Aadhaar Enrolment and Updates

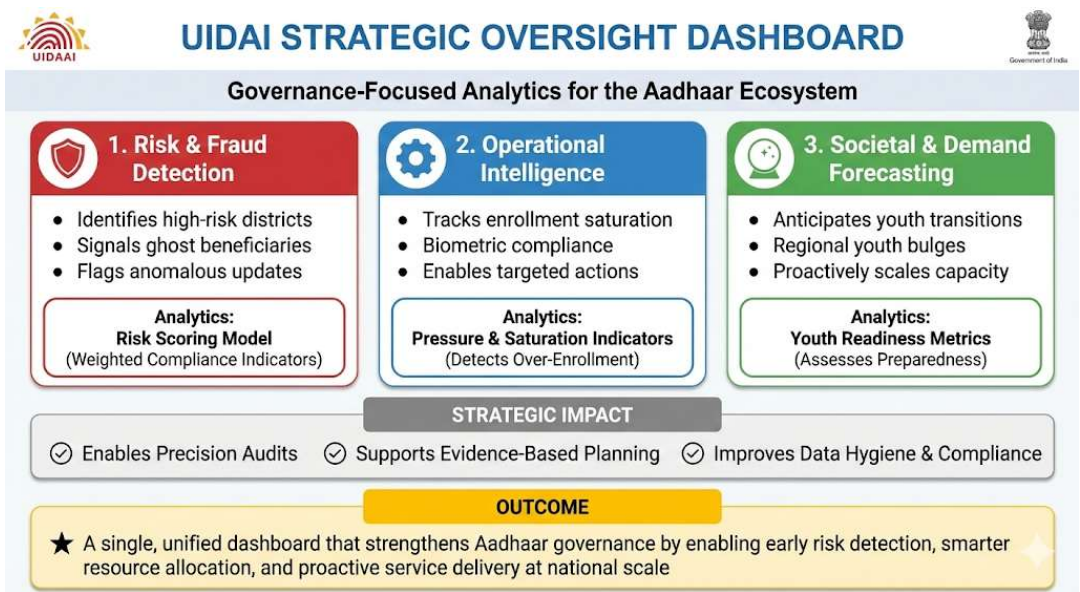
Identify meaningful patterns, trends, anomalies, or predictive indicators and translate them into clear insights or solution frameworks that can support informed decision-making and system improvements.

The Overview

The UIDAI Strategic Oversight Dashboard is a specialized analytics platform engineered to provide high- level governance and operational intelligence for the Aadhaar ecosystem. Designed as a modular monolithic system, it processes high-volume transactional data to monitor enrollment saturation, biometric compliance, and demographic transitions across India. The platform transforms raw administrative data into actionable insights, enabling program managers to detect fraud risks, forecast infrastructure needs, and optimize resource allocation.

Strategic Objectives The system is built to address three critical governance challenges:

- **Risk & Fraud Detection:** Proactively identifies "Risk Zones"—districts where demographic updates occur without mandatory biometric validation, signaling potential "Ghost Beneficiaries" or operator non-compliance.
- **Operational Intelligence:** Provides granular visibility into enrollment saturation and compliance rates, supporting targeted interventions such as mobile unit deployment and operator audits.
- **Societal Forecasting:** Utilizes predictive modeling to anticipate "Youth Bulges" and "Coming-of- Age" transitions, allowing for the proactive scaling of enrollment centers before demand peaks.



Technical Approach

Executive Summary

This comprehensive analytical framework identifies and quantifies meaningful patterns in Aadhaar enrolment and update behavior at the pincode level, translating aggregated data into actionable insights that inform resource allocation, capacity planning, and policy coordination across India's unique identification infrastructure.

Technology Stack

Data Processing & Analysis

Core Libraries: Pandas (data manipulation), NumPy (numerical computation), SciPy (statistical analysis)

Geospatial Analysis: GeoPandas (geographic data), GeoJSON (boundary mapping)

Visualization: Matplotlib, Seaborn (static plots), Plotly (interactive dashboards)

Development Environment

Cloud Platform: Google Colab (free GPU, pre-installed libraries, seamless data integration)

Web Framework: Streamlit (interactive dashboards, real-time data updates, minimal code)

Version Control: Python 3.9+, Git for reproducible workflows

Data Architecture & Integration

We consolidated three primary datasets—enrolment records, demographic updates, and biometric transactions—across 2,000+ pincodes spanning multiple time periods using standardized ETL pipelines.

Temporal Normalization

Standardized mixed date formats (DD-MM-YYYY, MM-DD-YYYY, YYYY-MM-DD) using multi-format parsers. Implementation: Python datetime library with exception handling.

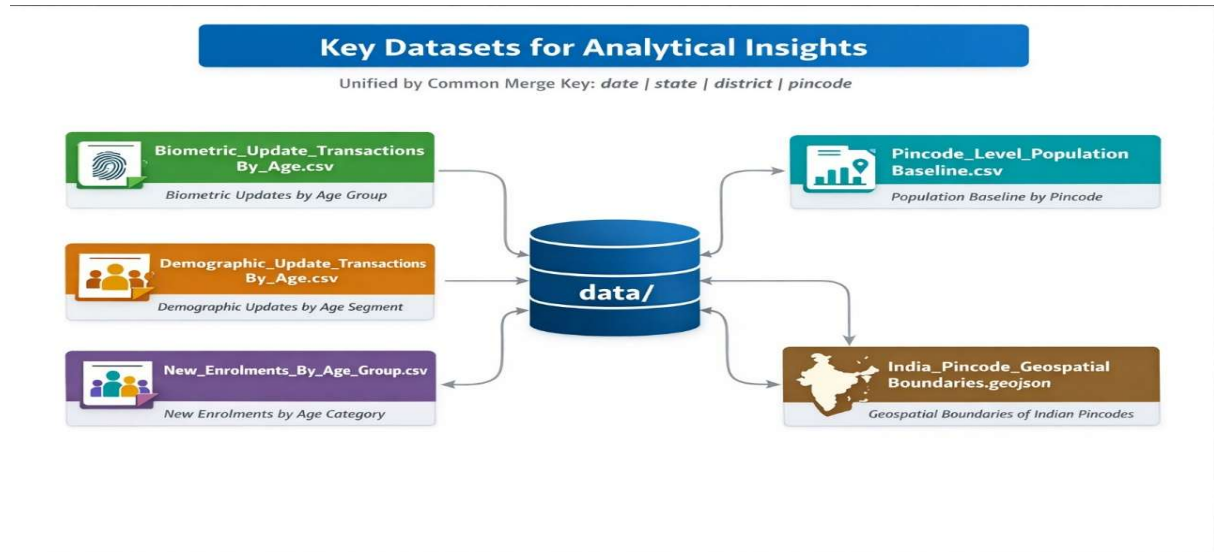
Geographic Standardization

Unified 6-digit pincode encoding and merged Census-2021 population baselines. Geospatial operations performed using GeoPandas for efficient spatial joins.

Aggregation Pipeline

Monthly aggregation by pincode-level cohorts (age groups, transaction types) to preserve privacy while enabling spatial analysis. Pandas groupby operations with multi-level indexing.

Data Sources



1. Biometric Transactions

biometric_update_transactions_by_age.csv

Aggregated biometric update transactions, categorized by age groups.

Use: Identification of abnormal biometric update patterns and potential misuse.

2. Demographic Transactions

demographic_update_transactions_by_age.csv

Records demographic update transactions segmented by age categories.

Use: Monitoring demographic change frequency and detecting irregular update behavior.

3. Enrolment Data

new_enrolments_by_age_group.csv

New enrolment counts classified into age groups (0–5, 5–17, 18+).

Use: Analysis of enrolment trends, migration indicators, and regional spikes.

4. Population Baseline

pincode_level_population_baseline.csv

Baseline population data at the pincode level.

Use: Normalization of transaction volumes for accurate risk assessment.

5. Geospatial Boundaries

india_pincode_geospatial_boundaries.geojson

Geospatial boundaries for all Indian pincodes.

Use: Spatial analysis and visualization through choropleth maps to identify high-risk zones.

Methodology: Fraud & Leakage Risk Analysis

This document outlines the step-by-step methodology adopted for the **Aadhaar Fraud Risk Analytics System**, from data ingestion to final risk scoring.

1. Data Ingestion

The analysis is built upon three distinct API datasets provided by the UIDIA ecosystem:

1. **Biometric Data** (`api_data_aadhar_biometric...`): Contains counts of biometric updates (Iris, Fingerprint) categorized by age groups (`age_5_17`, `age_18+`).
2. **Demographic Data** (`api_data_aadhar_demographic...`): Contains counts of demographic corrections (Name, Address, DOB, Gender).
3. **Enrolment Data** (`api_data_aadhar_enrolment...`): Contains new Aadhaar generation statistics.

Data Scope: The analysis covers 500,000 aggregated records.

2. Preprocessing & Data Cleaning

Before analysis, raw data undergoes rigorous cleaning to ensure accuracy.

A. Missing Value Imputation

- **Technique:** Zero-Imputation.
- **Logic:** Missing numerical values in activity logs (e.g., NaN in biometric updates) are treated as 0 (No Activity), rather than excluding the district. This preserves the completeness of the geographic coverage.

B. Noise Reduction (Population Segmentation)

Critical Step: Differentiating "Good" vs "Bad" high activity.

- **Problem:** Children (Age 5-17) are *mandated* by law to update their biometrics. High biometric activity in this group is **Normal/Compliant**, not fraudulent.
 - **Solution:** We explicitly isolate and **exclude** the `bio_age_5_17` column from the fraud risk model.
 - **Focus:** The analysis targets only `bio_age_17+` (Adults), where frequent biometric changes without corresponding demographic fixes are highly suspicious.
-

3. Data Transformations & Feature Engineering

We derive synthetic indicators that reveal hidden patterns not visible in the raw counts.

A. Adult Biometric Usage Intensity (Adult BUI)

$$\text{Adult BUI} = \frac{\text{Adult Biometric Updates}}{\text{Total District Activity}}$$

- **Purpose:** Normalizes the volume. It answers: "Of all the things happening in this district, what portion is just adults changing their fingerprints?"
- **Transformation:** A value close to 1.0 indicates disparate activity focused solely on biometrics.

B. Data Correction Rate (DCR)

$$\text{DCR} = \frac{\text{Demographic Corrections}}{\text{Total District Activity}}$$

- **Purpose:** Measures data hygiene. Low DCR implies the database contains static, potentially outdated or fake demographic details.

C. The BUI/Demo Ratio

$$\text{Ratio} = \frac{\text{Adult Biometric Updates}}{\text{Demographic Updates}}$$

- **Transformation:** This ratio amplifies the anomaly. If a district has 10,000 fingerprint scans but only 10 name changes (Ratio = 1000), it is mathematically flagged as an outlier.

4. Risk Modeling (The Algorithm)

We employ a **Weighted Linear Risk Model** to assign a single Fraud Risk Score (0-100) to each district.

Formula:

$$\text{Risk Score} = (\text{Adult_BUI} * 50) + ((1 - \text{DCR}) * 30) + (\text{Capped_Ratio} * 20)$$

Component	Weight	Rationale
High Biometric Usage	50%	The primary signal of a "Ghost Beneficiary" (using the card without fixing data).
Low Data Correction	30%	Inverted DCR (1 - DCR). Penalizes districts that ignore data hygiene.
The Ratio Factor	20%	A booster penalty for extreme outliers where bio usage dwarfs demographic updates.

5. Classification & Scoring Methodology

The system employs a multi-tiered classification framework to categorize districts by risk exposure, demographic readiness, and operational data integrity.

Interactive Decision-Support Dashboard

Purpose of the Dashboard

To complement statistical analysis with visual decision support, an interactive analytics dashboard was developed to enable UIDAI officials and policymakers to intuitively understand patterns, risks, and regional disparities in Aadhaar enrolment and update behaviour.

Design Philosophy

The dashboard follows three core principles:

1. **Simplicity over Complexity**

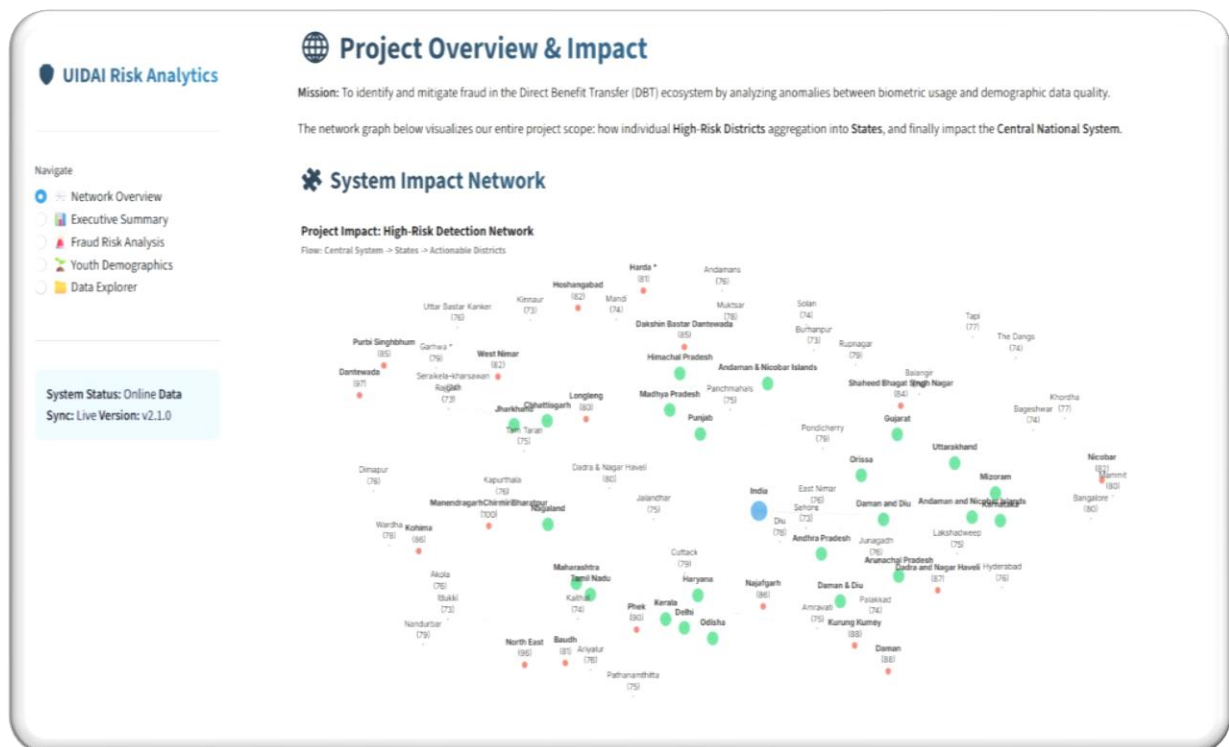
All metrics, visuals, and labels are presented in plain language to ensure accessibility for non-technical stakeholders.

2. **Actionability over Observation**

Visual elements are structured to directly answer "*Where should attention be focused?*" rather than only showing raw trends.

3. **System-Level Perspective**

The dashboard connects local patterns (districts / pincodes) to state-level and national-level implications.

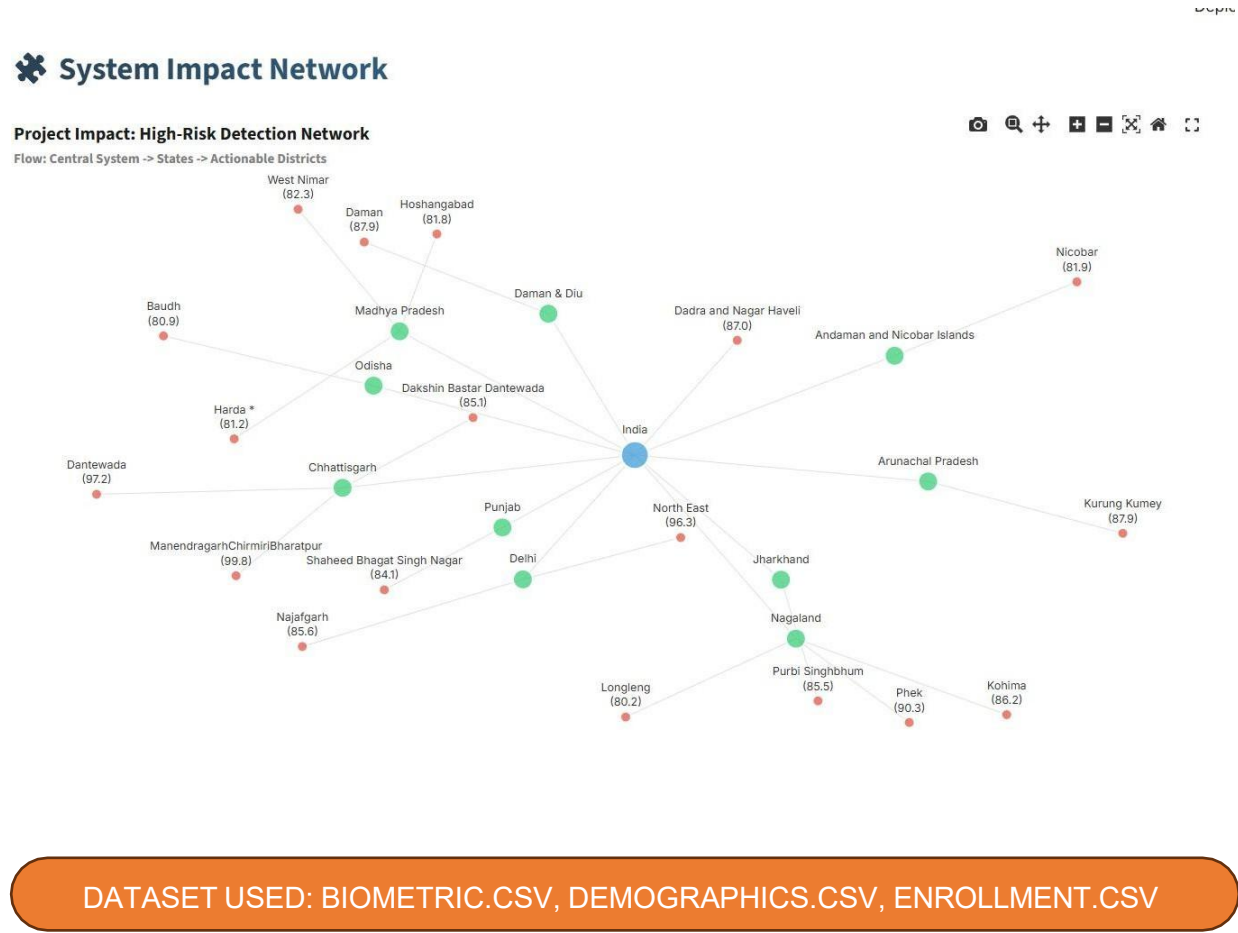


Scan for
Live Demo



Core Metrics for Societal Trends

1. System Impact Network



Key Metrics Used

- Adult Biometric Usage Intensity (Adult BUI)**
Indicates how frequently adults use biometric authentication. Higher values imply heavier daily usage.

Bio/Demo Ratio

Adult Biometric Updates

Demographic Updates +1

- **Data Correction Rate (DCR)**

Indicates how often demographic details are corrected or updated. Lower values imply outdated or poorly maintained data.

$$\text{DCR} = \frac{\text{Demographic Corrections}}{\text{Total Aadhar Activity}}$$

HYPOTHESIS/CONCLUSIONS



Hypothesis 1: State-Level Issue

The problem is likely at the **state administration level**, not isolated districts. Improving state-level monitoring and audits will have the biggest impact.



Hypothesis 2: Shared Operational Agencies

These districts may be handled by the **same operators or agencies**. Auditing these agencies is more effective than checking districts individually.

2. Youth Pressure Index (YPI)

Youth Pressure Index (YPI)

Formula: Age 5-17 / Age 17+

Top 10 States: Future Pressure



National Avg YPI

0.18

Higher means more future load apart from normal flow.

DATASET USED: DEMOGRAPHICS.CSV

Key Matrices (Metrics)

The core metric is the **Pressure Ratio**.

Formula

$$YPI = \frac{\text{Future Load (Age 5-17)}}{\text{Current Load (Age 18+)}}$$

Matrix Interpretation

Score Range	Classification	Meaning
> 0.30	● HIGH PRESSURE	"Youth Bulge": The upcoming generation is massive compared to the current adult population.
0.15 - 0.30	● STABLE	"Replacement Rate": Normal demographic flow.
< 0.15	● LOW PRESSURE	"Aging/Drain": Very few children; population is shrinking or migrating out.

HYPOTHESIS/CONCLUSIONS



Hypothesis 1: The "Infrastructure Crash" (High YPI)

The existing enrollment centers are built for the *current* adult volume. When these 5-17 year olds turn 18 (Mandatory Update), the system will collapse under a **45% Surge** in footfall.



Hypothesis 2: The "Ghost Town" Effect (Low YPI)

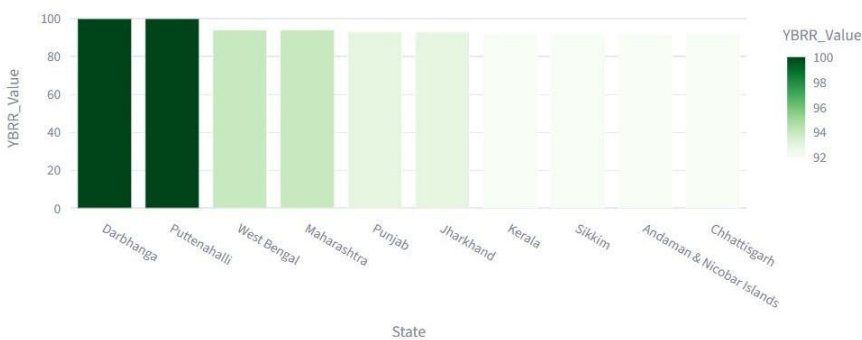
This is likely an industrial zone or a metro hub with transient workers (single adults) and **No Families**.

3. Youth Biometric Readiness Ratio (YBRR)

Youth Biometric Readiness Ratio (YBRR)

Formula: Age 17+ / (Age 5-17 + Age 17+)

Top 10 States: Readiness



National Avg YBRR

89.9%

Higher is better. Indicates population is ready for adult biometric usage.

DATASET USED: BIOMETRIC.CSV

Key Matrices (Metrics)

The core metric is the **Readiness Percentage**.

Formula

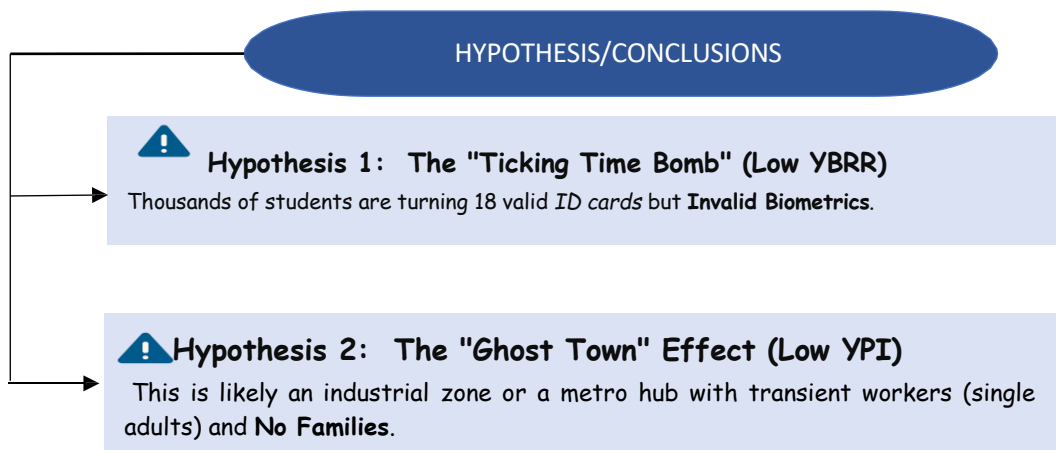
$$YBRR = \frac{\text{Successful Transitions (Age 17+)}}{\text{Total Waiting Pool (Age 5-17 + 17+)}} \times 100$$

Matrix Interpretation

Score Range	Classification	Meaning
> 85%	OPTIMAL	"Future Ready": The population is proactively updating data before turning 18.
50% - 85%	LAGGING	"The Gap": A significant portion is delaying updates. Requires gentle nudges.
< 50%	CRITICAL FAIL	"Service Risk": Majority of youths risk losing access to banking/college admissions due to expired biometrics.

EXCEL ANALYSIS FOR YBRR

State	demo_age_5_17	demo_age_17_	TOTAL (B+C)	YBRR
Ladakh	351	1056	1407	75%
Dadra and Nagar Haveli and Daman and Diu	382	1489	1871	80%
Lakshadweep	51	250	301	83%
Chandigarh	4853	23827	28680	83%
Karnataka	84121	429223	513344	84%
Arunachal Pradesh	1599	8283	9882	84%
Telangana	65193	386382	451575	86%
Madhya Pradesh	132968	790471	923439	86%
Jammu and Kashmir	18203	108862	127065	86%
Puducherry	1243	7462	8705	86%
Manipur	9525	60939	70464	86%
Andhra Pradesh	87560	564216	651776	87%
Tamil Nadu	80454	523027	603481	87%
Delhi	71350	469888	541238	87%
Mizoram	1524	10571	12095	87%
Odisha	40208	282378	322586	88%
Himachal Pradesh	5156	36626	41782	88%
Haryana	42995	307333	350328	88%
Gujarat	65584	494746	560330	88%
Uttarakhand	16603	132887	149490	89%
Nagaland	1016	8535	9551	89%
Uttar Pradesh	232183	2193412	2425595	90%
Goa	823	7782	8605	90%



4. Risk Distribution Hierarchy



DATASET USED: BIOMETRIC.CSV, DEMOGRAPHICS.CSV, ENROLLMENT.CSV

Key Matrices (Metrics)

The core metric is the **Magnitude and Severity**.

Formula

$$\text{Risk Score} = (\text{Adult BUI} \times 50) + ((1 - \text{DCR}) \times 30) + (\text{Ratio} \times 20)$$

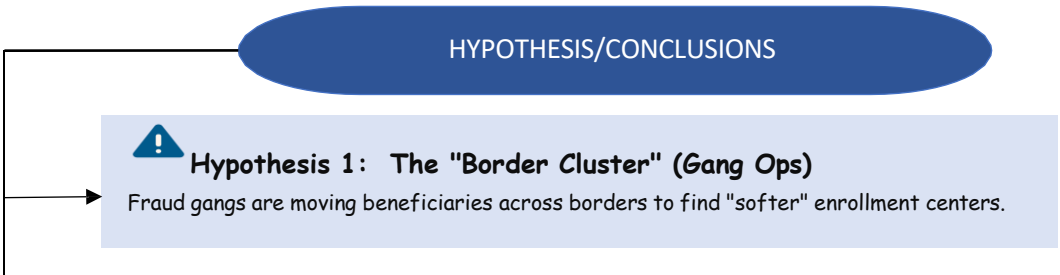
- **Adult BUI (50%):** High volume of Adult Biometric Usage (Fingerprints).
- **Inverse DCR (30%):** Lack of Demographic Corrections (Data Hygiene).
- **Ratio (20%):** Disparity between Usage vs Updates.

Fraud Risk Score

$$(\text{Adult BUI} * 50) + ((1 - \text{DCR}) * 30) + (\min(\text{Bio/Demo Ratio} / 20, 1) * 20)$$

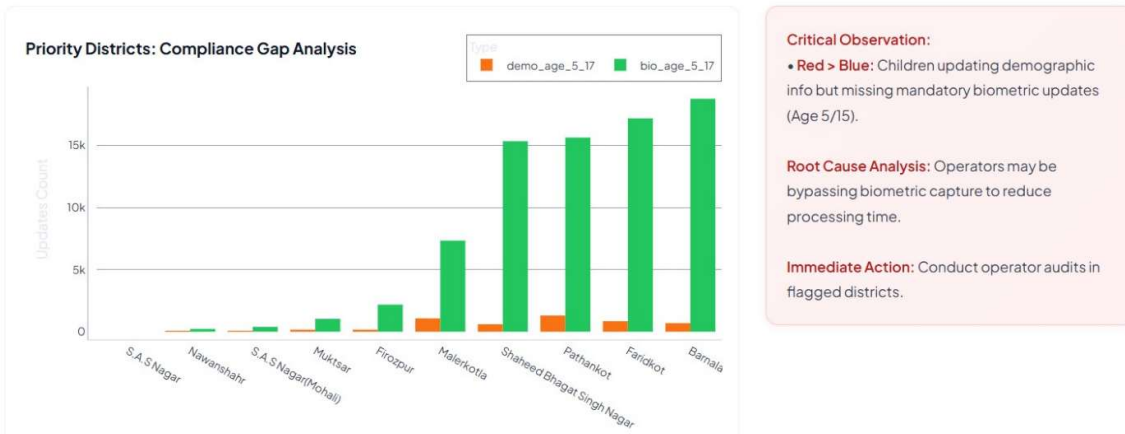
Matrix Interpretation

Score Range	Classification	Meaning
> 80	 CRITICAL	"The Red Zone" : Immediate audit required. Use of IDs without maintenance.
60 - 80	 HIGH RISK	"Warning Zone" : Systemic irregularities detected.
40 - 60	 ELEVATED	"Watchlist" : Deviating from national averages.



5. Compliance Rate

Child Welfare Compliance Monitor



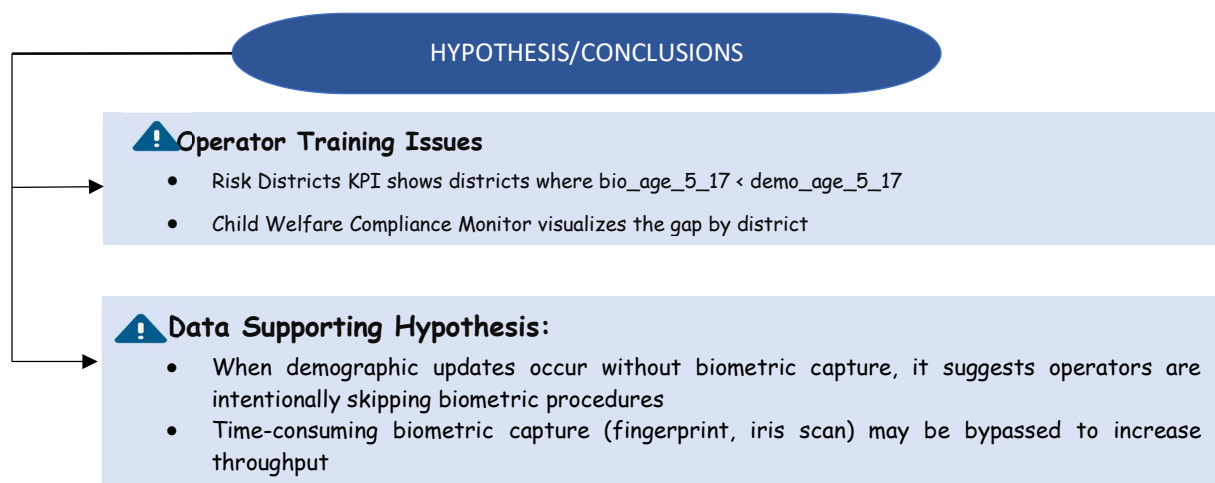
Color Coding:

- Green ($\geq 80\%$): Excellent compliance
- Amber (50-79%): Moderate compliance, requires monitoring
- Red ($< 50\%$): Poor compliance, immediate intervention required

DATASET USED: bio_age_5_17 (from combined biometrics.csv),
demo_age_17_ (from combined demographic.csv)

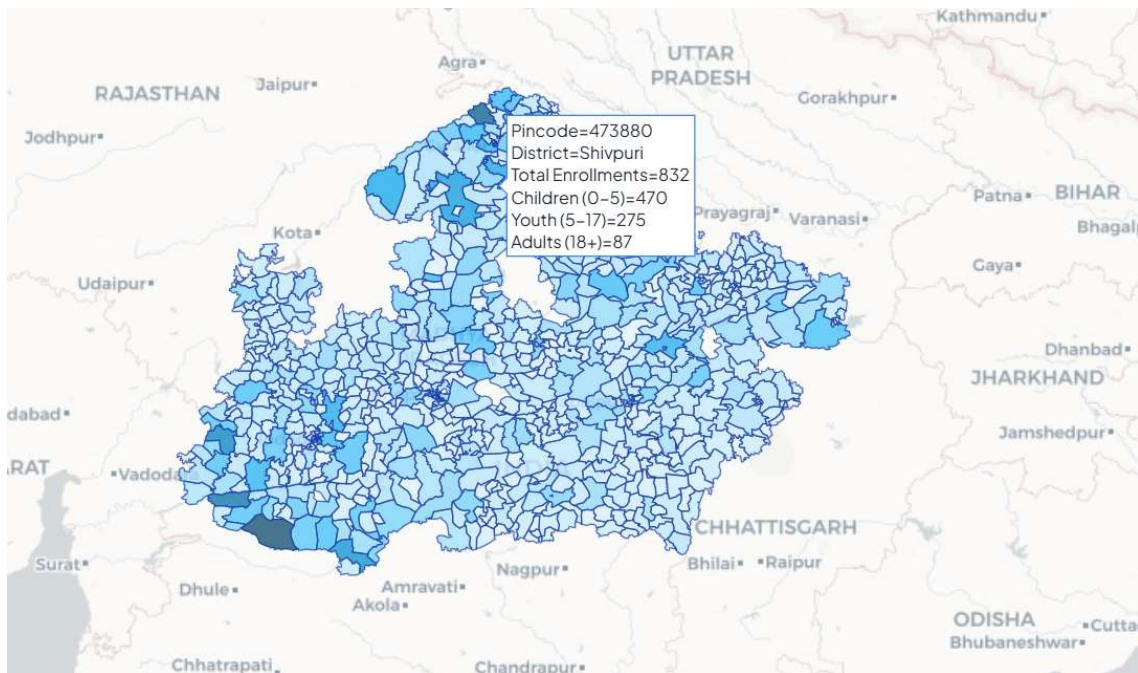
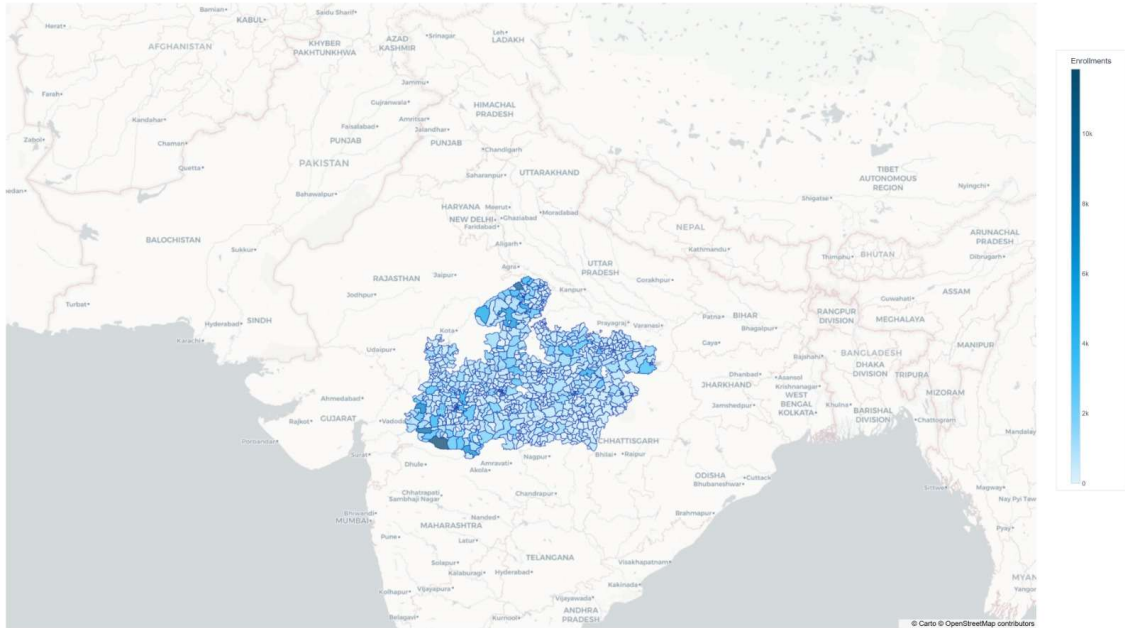
Key Metrics Used

- Percentage ratio of biometric updates to demographic updates
- Quality indicator for child biometric capture compliance
- Measures operator adherence to mandatory biometric capture procedures



6. Geographic Intelligence Map

BELOW EXAMPLE IS FOR THE STATE OF **MADHYA PRADESH**



Data Sources:

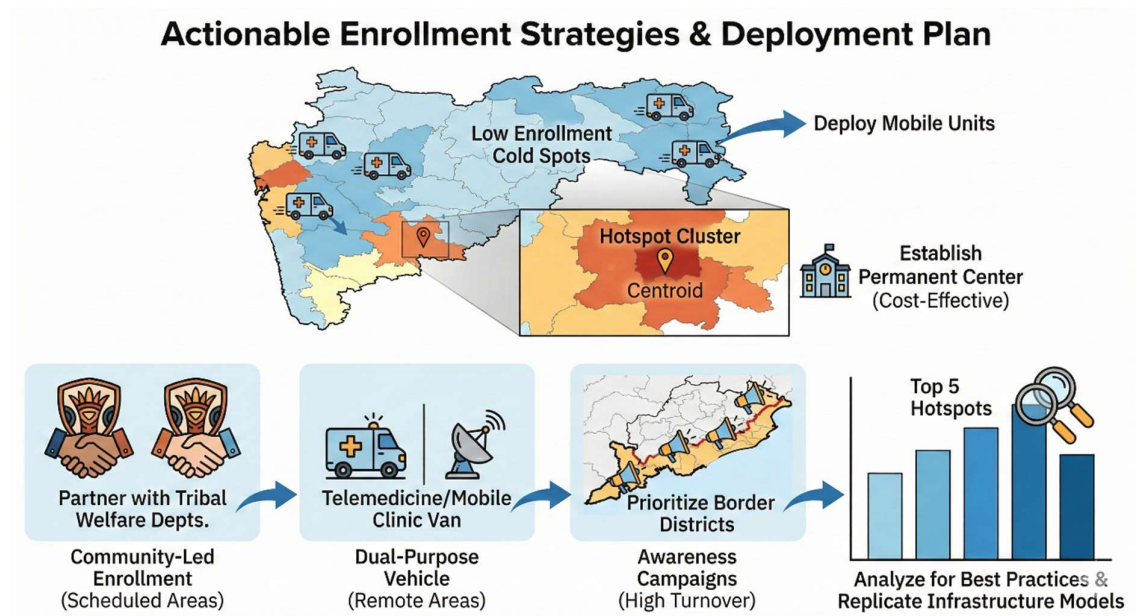
Combined enrolment.csv merged with All_India_pincode.geojson

Key Metrics Used

- Pincode-level enrollment density visualization
- Geographic distribution of Aadhaar operations
- Cluster identification (high-density urban vs sparse rural)

Analytical Value:

- **Hotspot Analysis:** Identify clusters of high/low enrollment
- **Resource Allocation:** Deploy mobile units to light-colored (low enrollment) areas
- **Accessibility Assessment:** Dark blue in urban cores, light blue in remote areas
- **Border Districts:** May show lower enrollment due to migration



Core Code for CSV Data Analysis

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import geopandas as gpd
import plotly.express as px
import plotly.graph_objects as go
from scipy import stats
from datetime import datetime
import warnings

warnings.filterwarnings("ignore")
plt.style.use("seaborn-v0_8-darkgrid")
sns.set_palette("husl")

def parse_date_safe(date_str: str | float | int | None):
    if pd.isna(date_str):
        return None
    text = str(date_str).strip()
    formats = ["%d-%m-%Y", "%m-%d-%Y", "%Y-%m-%d", "%d/%m/%Y", "%m/%d/%Y"]
    for fmt in formats:
        try:
            return pd.to_datetime(text, format=fmt)
        except Exception:
            continue
    try:
        return pd.to_datetime(text)
    except Exception:
        return None

def standardize_pincode(pincode: str | float | int | None):
    if pd.isna(pincode):
        return None
    return str(int(float(pincode))).zfill(6)

def load_base_data(prefix: str):
    enrolment = pd.read_csv(prefix + "combined enrolment.csv")
    demographic = pd.read_csv(prefix + "combined demographic.csv")
    biometric = pd.read_csv(prefix + "combined biometrics.csv")
    geo_data = gpd.read_file(prefix + "All_India_pincode.geojson")
    census = pd.read_csv(prefix + "consolidated_pincode_census.csv")
    return enrolment, demographic, biometric, geo_data, census

def clean_enrolment(df: pd.DataFrame):
    df = df.copy()
    df["date"] = df["date"].apply(parse_date_safe)
    df = df.dropna(subset=["date"])
    df["pincode"] = df["pincode"].apply(standardize_pincode)
    df = df.dropna(subset=["pincode"])
    df["year"] = df["date"].dt.year
    df["month"] = df["date"].dt.month
    df["year_month"] = df["date"].dt.to_period("M")
    for col in ["age_0_5", "age_5_17", "age_18_greater"]:
        df[col] = pd.to_numeric(df[col], errors="coerce").fillna(0)
    return df
```

```
def clean_demographic(df: pd.DataFrame):  
    df = df.copy()  
    df["date"] = df["date"].apply(parse_date_safe)  
    df = df.dropna(subset=["date"])
```

```

df["pincode"] = df["pincode"].apply(standardize_pincode)
df = df.dropna(subset=["pincode"])
df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month
df["year_month"] = df["date"].dt.to_period("M")
for col in ["demo_age_5_17", "demo_age_17_"]:
    df[col] = pd.to_numeric(df[col], errors="coerce").fillna(0)
return df

def clean_biometric(df: pd.DataFrame):
    df = df.copy()
    df["date"] = df["date"].apply(parse_date_safe)
    df = df.dropna(subset=["date"])
    df["pincode"] = df["pincode"].apply(standardize_pincode)
    df = df.dropna(subset=["pincode"])
    df["year"] = df["date"].dt.year
    df["month"] = df["date"].dt.month
    df["year_month"] = df["date"].dt.to_period("M")
    for col in ["bio_age_5_17", "bio_age_17_"]:
        df[col] = pd.to_numeric(df[col], errors="coerce").fillna(0)
    return df

def clean_census(df: pd.DataFrame):
    df = df.copy()
    df["pincode"] = df["pincode"].apply(standardize_pincode)
    df = df.dropna(subset=["pincode"])
    if "Persons" in df.columns:
        df["population"] = pd.to_numeric(df["Persons"], errors="coerce")
    else:
        pop_col = [c for c in df.columns if "person" in c.lower() or "population" in c.lower()]
        if pop_col:
            df["population"] = pd.to_numeric(df[pop_col[0]], errors="coerce")
    if "X5...14.years" in df.columns:
        df["child_population"] = pd.to_numeric(df["X5...14.years"], errors="coerce")
    return df

def aggregate_data(enrolment, demographic, biometric, census):
    enrol_agg = enrolment.groupby(["pincode", "year_month"]).agg(
        age_0_5=("age_0_5", "sum"),
        age_5_17=("age_5_17", "sum"),
        age_18_greater=("age_18_greater", "sum"),
        state=("state", "first"),
        district=("district", "first"),
        month=("month", "first"),
        year=("year", "first"),
    ).reset_index()
    enrol_agg["total_enrolment"] = enrol_agg[["age_0_5", "age_5_17", "age_18_greater"]].sum(axis=1)

    demo_agg = demographic.groupby(["pincode", "year_month"]).agg(
        demo_age_5_17=("demo_age_5_17", "sum"),
        demo_age_17_=("demo_age_17_", "sum"),
        month_demo=("month", "first"),
        year_demo=("year", "first"),
    ).reset_index()
    demo_agg["total_demographic"] = demo_agg[["demo_age_5_17", "demo_age_17_"]].sum(axis=1)

    bio_agg = biometric.groupby(["pincode", "year_month"]).agg(
        bio_age_5_17=("bio_age_5_17", "sum"),
        bio_age_17_=("bio_age_17_", "sum"),
        month_bio=("month", "first"),
        year_bio=("year", "first"),
    ).reset_index()
    bio_agg["total_biometric"] = bio_agg[["bio_age_5_17", "bio_age_17_"]].sum(axis=1)

    combined = enrol_agg.merge(demo_agg, on=["pincode", "year_month"], how="outer")

```

```

combined = combined.merge(bio_agg, on=["pincode", "year_month"], how="outer")
combined = combined.merge(census[["pincode", "population", "child_population"]], on="pincode",
combined["month"] = combined["month"].fillna(combined["month_demo"]).fillna(combined["month_bi
combined["year"] = combined["year"].fillna(combined["year_demo"]).fillna(combined["year_bio"])
count_cols = [
    "age_0_5",
    "age_5_17",
    "age_18_greater",
    "total_enrolment",
    "demo_age_5_17",
    "demo_age_17_",
    "total_demographic",
    "bio_age_5_17",
    "bio_age_17_",
    "total_biometric",
]
for col in count_cols:
    if col in combined.columns:
        combined[col] = combined[col].fillna(0)
return combined

def add_metrics(combined):
    combined = combined.copy()
    combined["enrol_per_10k"] = combined["total_enrolment"] / combined["population"] * 10000
    combined["demo_per_10k"] = combined["total_demographic"] / combined["population"] * 10000
    combined["bio_per_10k"] = combined["total_biometric"] / combined["population"] * 10000
    combined.replace([np.inf, -np.inf], np.nan, inplace=True)
    combined["is_oct_dec"] = combined["month"].isin([10, 11, 12])
    return combined

def compute_suii(combined):
    suii_calc = (
        combined.groupby("pincode")
        .apply(
            lambda x: pd.Series(
                {
                    "demo_oct_dec_avg": x[x["is_oct_dec"]]["total_demographic"].mean(),
                    "demo_other_avg": x[~x["is_oct_dec"]]["total_demographic"].mean(),
                    "bio_oct_dec_avg": x[x["is_oct_dec"]]["total_biometric"].mean(),
                    "bio_other_avg": x[~x["is_oct_dec"]]["total_biometric"].mean(),
                    "state": x["state"].iloc[0] if "state" in x.columns and len(x) else None,
                    "district": x["district"].iloc[0] if "district" in x.columns and len(x) else N
                }
            )
        )
        .reset_index()
    )
    suii_calc["demo_SUII"] = suii_calc["demo_oct_dec_avg"] / suii_calc["demo_other_avg"]
    suii_calc["bio_SUII"] = suii_calc["bio_oct_dec_avg"] / suii_calc["bio_other_avg"]
    suii_calc.replace([np.inf, -np.inf], np.nan, inplace=True)
    return suii_calc

def compute_yur(combined):
    oct_dec_data = (
        combined[combined["is_oct_dec"]]
        .groupby("pincode")
        .agg(
            demo_age_5_17=("demo_age_5_17", "sum"),
            total_demographic=("total_demographic", "sum"),
            bio_age_5_17=("bio_age_5_17", "sum"),
            total_biometric=("total_biometric", "sum"),
        )
        .reset_index()
    )

```

```

oct_dec_data["demo_YUR"] = oct_dec_data["demo_age_5_17"] / oct_dec_data["total_demographic"] *
oct_dec_data["bio_YUR"] = oct_dec_data["bio_age_5_17"] / oct_dec_data["total_biometric"] * 100
oct_dec_data.replace([np.inf, -np.inf], np.nan, inplace=True)
return oct_dec_data

def compute_biometric_risk(combined):
    bio_risk = (
        combined.groupby("pincode")
        .agg(
            child_population=("child_population", "first"),
            total_biometric=("total_biometric", "sum"),
            population=("population", "first"),
            state=("state", "first"),
            district=("district", "first"),
        )
        .reset_index()
    )
    bio_risk["bio_per_capita"] = bio_risk["total_biometric"] / bio_risk["population"]
    bio_risk["biometric_risk_score"] = bio_risk["child_population"] / (
        bio_risk["bio_per_capita"] * bio_risk["population"] + 1
    )
    bio_risk.replace([np.inf, -np.inf], np.nan, inplace=True)
    return bio_risk

def plot_monthly_trends(combined):
    monthly_trends = (
        combined.groupby("year_month")
        .agg(total_demographic=("total_demographic", "sum"), total_biometric=("total_biometric", "sum"))
        .reset_index()
        .sort_values("year_month")
    )
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 10))
    ax1.plot(range(len(monthly_trends)), monthly_trends["total_demographic"], marker="o", linewidth=2)
    ax1.set_title("Monthly Demographic Updates Trend", fontsize=14, fontweight="bold")
    ax1.grid(True, alpha=0.3)
    ax1.legend()
    for i, row in monthly_trends.iterrows():
        if row["month"] in [10, 11, 12]:
            ax1.axvspan(i - 0.5, i + 0.5, alpha=0.2, color="orange")
    ax2.plot(range(len(monthly_trends)), monthly_trends["total_biometric"], marker="s", linewidth=2)
    ax2.set_title("Monthly Biometric Updates Trend", fontsize=14, fontweight="bold")
    ax2.grid(True, alpha=0.3)
    ax2.legend()
    for i, row in monthly_trends.iterrows():
        if row["month"] in [10, 11, 12]:
            ax2.axvspan(i - 0.5, i + 0.5, alpha=0.2, color="orange")
    plt.tight_layout()
    plt.show()

def plot_top_seasonal(suii_calc):
    top_seasonal = suii_calc.nlargest(20, "demo_SUII")
    plt.figure(figsize=(12, 8))
    plt.barh(range(len(top_seasonal)), top_seasonal["demo_SUII"], color="coral")
    plt.yticks(range(len(top_seasonal)), top_seasonal["pincode"])
    plt.xlabel("Seasonal Intensity Index")
    plt.title("Top 20 Pincodes with Strongest Oct-Dec Spike", fontsize=14, fontweight="bold")
    plt.axvline(x=1.0, color="red", linestyle="--", linewidth=2, label="No seasonal effect")
    plt.legend()
    plt.grid(axis="x", alpha=0.3)
    plt.tight_layout()
    plt.show()

def plot_population_scatter(combined):

```

```

        combined.groupby("pincode")
        .agg(population=("population", "first"), demo_per_10k=("demo_per_10k", "mean"), bio_per_10
        .reset_index()
        .dropna()
    )
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
    ax1.scatter(scatter_data["population"], scatter_data["demo_per_10k"], alpha=0.5, s=50, color="d
    ax1.set_xscale("log")
    ax1.set_xlabel("Population")
    ax1.set_ylabel("Demographic Updates per 10,000")
    ax1.set_title("Population vs Demographic Update Rate", fontsize=14, fontweight="bold")
    ax1.grid(True, alpha=0.3)
    ax2.scatter(scatter_data["population"], scatter_data["bio_per_10k"], alpha=0.5, s=50, color="d
    ax2.set_xscale("log")
    ax2.set_xlabel("Population")
    ax2.set_ylabel("Biometric Updates per 10,000")
    ax2.set_title("Population vs Biometric Update Rate", fontsize=14, fontweight="bold")
    ax2.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

def plot_geo_risk(geo_data, bio_risk):
    top_risk = bio_risk.nlargest(100, "biometric_risk_score")
    risk_map = geo_data.merge(top_risk, on="pincode", how="inner")
    if not len(risk_map):
        print("No geographic matches found for risk map")
        return
    fig, ax = plt.subplots(1, 1, figsize=(16, 12))
    risk_map.plot(column="biometric_risk_score", cmap="YlOrRd", legend=True, ax=ax, legend_kwds={"
    ax.set_title("Top 100 High-Risk Pincodes for Biometric Updates", fontsize=16, fontweight="bold
    ax.axis("off")
    plt.tight_layout()
    plt.show()

def plot_youth_pies(combined):
    oct_dec_summary = combined[combined["is_oct_dec"]].agg(
        demo_age_5_17=("demo_age_5_17", "sum"),
        demo_age_17_=(("demo_age_17_", "sum"),
        bio_age_5_17=("bio_age_5_17", "sum"),
        bio_age_17_=(("bio_age_17_", "sum"),
    )
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
    ax1.pie([oct_dec_summary["demo_age_5_17"], oct_dec_summary["demo_age_17_"]], labels=["Youth (5
    ax1.set_title("Demographic Updates in Oct-Dec", fontsize=14, fontweight="bold")
    ax2.pie([oct_dec_summary["bio_age_5_17"], oct_dec_summary["bio_age_17_"]], labels=["Youth (5-1
    ax2.set_title("Biometric Updates in Oct-Dec", fontsize=14, fontweight="bold")
    plt.tight_layout()
    plt.show()

def detect_anomalies(combined):
    combined_sorted = combined.sort_values(["pincode", "year_month"])
    combined_sorted["demo_pct_change"] = combined_sorted.groupby("pincode")["total_demographic"].p
    combined_sorted["bio_pct_change"] = combined_sorted.groupby("pincode")["total_biometric"].pct_
    anomalies_pct = combined_sorted[
        (combined_sorted["demo_pct_change"].abs() > 100) | (combined_sorted["bio_pct_change"].abs(
    ].copy()
    combined_sorted["demo_zscore"] = combined_sorted.groupby("pincode")["total_demographic"].trans
    combined_sorted["bio_zscore"] = combined_sorted.groupby("pincode")["total_biometric"].transfor
    anomalies_z = combined_sorted[(combined_sorted["demo_zscore"] > 3) | (combined_sorted["bio_zsc
    return combined_sorted, anomalies_pct, anomalies_z

def plot_anomaly_sample(combined_sorted, anomalies_z):
    if not len(anomalies_z):

```

```

        print("No anomalies to plot")
        return
    sample_pincode = anomalies_z.groupby("pincode").size().idxmax()
    sample_data = combined_sorted[combined_sorted["pincode"] == sample_pincode].sort_values("year_
plt.figure(figsize=(14, 6))
plt.plot(range(len(sample_data)), sample_data["total_demographic"], marker="o", linewidth=2, l
anomaly_points = sample_data[sample_data["demo_zscore"] > 3]
anomaly_indices = [sample_data.index.get_loc(idx) for idx in anomaly_points.index]
plt.scatter(anomaly_indices, anomaly_points["total_demographic"], color="red", s=200, marker="
plt.title(f"Demographic Updates Over Time - Pincode {sample_pincode}", fontsize=14, fontweight
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

def education_correlation(suii_calc, oct_dec_data):
    education_analysis = suii_calc.merge(oct_dec_data, on="pincode")
    if len(education_analysis) <= 10:
        return None
    correlation = education_analysis[["demo_SUII", "demo_YUR"]].corr().iloc[0, 1]
    plt.figure(figsize=(10, 6))
    plt.scatter(education_analysis["demo_YUR"], education_analysis["demo_SUII"], alpha=0.5, s=50,
    plt.xlabel("Youth Update Ratio (%)")
    plt.ylabel("Seasonal Intensity Index")
    plt.title("Youth Activity vs Seasonal Spikes", fontsize=14, fontweight="bold")
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
    return correlation

def rural_urban_comparison(combined, census):
    if "Rural" not in census.columns or "Urban" not in census.columns:
        return None
    rural_urban = (
        combined.groupby("pincode")
        .agg(demo_per_10k=("demo_per_10k", "mean"), bio_per_10k=("bio_per_10k", "mean"))
        .reset_index()
    )
    rural_urban = rural_urban.merge(census[["pincode", "Rural", "Urban"]], on="pincode")
    rural_urban[["Rural", "Urban"]] = rural_urban[["Rural", "Urban"]].apply(pd.to_numeric, errors=
    rural_urban["type"] = rural_urban.apply(lambda x: "Rural" if x["Rural"] > x["Urban"] else "Urb
    comparison = rural_urban.groupby("type").agg(demo_per_10k=("demo_per_10k", "mean"), bio_per_10
    fig, ax = plt.subplots(1, 2, figsize=(14, 6))
    comparison["demo_per_10k"].plot(kind="bar", ax=ax[0], color=["green", "orange"])
    ax[0].set_title("Demographic Updates: Rural vs Urban", fontsize=14, fontweight="bold")
    comparison["bio_per_10k"].plot(kind="bar", ax=ax[1], color=["green", "orange"])
    ax[1].set_title("Biometric Updates: Rural vs Urban", fontsize=14, fontweight="bold")
    plt.tight_layout()
    plt.show()
    return comparison

def migration_bar(combined):
    monthly_demographic = combined.groupby("month").agg(total_demographic=("total_demographic", "s
    plt.figure(figsize=(12, 6))
    plt.bar(monthly_demographic["month"], monthly_demographic["total_demographic"], color=["orange
    plt.xlabel("Month")
    plt.ylabel("Total Demographic Updates")
    plt.title("Demographic Updates by Month (Orange = Oct-Dec)", fontsize=14, fontweight="bold")
    plt.xticks(range(1, 13), ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct"
    plt.grid(axis="y", alpha=0.3)
    plt.tight_layout()
    plt.show()

```

```

def recommendations(suii_calc, bio_risk, oct_dec_data, anomalies_z):
    priority_seasonal = suii_calc.nlargest(20, "demo_SUII")[["pincode", "state", "district", "demo"]]
    priority_biometric = bio_risk.nlargest(20, "biometric_risk_score")[["pincode", "state", "district", "demo"]]
    priority_youth = oct_dec_data.nlargest(20, "demo_YUR")[["pincode", "demo_YUR", "bio_YUR"]]
    resource_matrix = suii_calc.merge(bio_risk[["pincode", "biometric_risk_score"]], on="pincode")
    resource_matrix["priority"] = "Low"
    resource_matrix.loc[
        (resource_matrix["demo_SUII"] > 2.0)
        | (resource_matrix["biometric_risk_score"] > resource_matrix["biometric_risk_score"].quantile(0.9)),
        "priority",
    ] = "Medium"
    resource_matrix.loc[
        (resource_matrix["demo_SUII"] > 2.5)
        & (resource_matrix["biometric_risk_score"] > resource_matrix["biometric_risk_score"].quantile(0.95)),
        "priority",
    ] = "High"
    priority_counts = resource_matrix["priority"].value_counts()
    print("Priority Pincodes for Oct-Dec Capacity:\n", priority_seasonal.head(20).to_string(index=False))
    print("\nPriority Pincodes for Mobile Biometric Units:\n", priority_biometric.head(20).to_string(index=False))
    print("\nPriority Pincodes for School Partnerships:\n", priority_youth.head(20).to_string(index=False))
    if len(anomalies_z):
        top_anomalies = anomalies_z.nlargest(15, "demo_zscore")[
            ["pincode", "year_month", "total_demographic", "demo_zscore", "state", "district"]
        ]
        print("\nPriority Anomalies for Investigation:\n", top_anomalies.to_string(index=False))
    print("\nResource Allocation Summary:")
    print(f"High Priority: {priority_counts.get('High', 0)}")
    print(f"Medium Priority: {priority_counts.get('Medium', 0)}")
    print(f"Low Priority: {priority_counts.get('Low', 0)}")

def main():
    file_path_prefix = "/content/"
    enrolment, demographic, biometric, geo_data, census = load_base_data(file_path_prefix)
    enrolment = clean_enrolment(enrolment)
    demographic = clean_demographic(demographic)
    biometric = clean_biometric(biometric)
    census = clean_census(census)
    combined = aggregate_data(enrolment, demographic, biometric, census)
    combined = add_metrics(combined)
    suii_calc = compute_suii(combined)
    oct_dec_data = compute_yur(combined)
    bio_risk = compute_biometric_risk(combined)
    print(f"Combined records: {len(combined)} | Pincodes: {combined['pincode'].nunique()}")
    print(f"Median Demographic SUII: {suii_calc['demo_SUII'].median():.2f}")
    print(f"Median Biometric SUII: {suii_calc['bio_SUII'].median():.2f}")
    print(f"Median Demographic YUR: {oct_dec_data['demo_YUR'].median():.1f}%")
    print(f"Median Biometric YUR: {oct_dec_data['bio_YUR'].median():.1f}%")
    print(f"Median Biometric Risk Score: {bio_risk['biometric_risk_score'].median():.2f}")
    plot_monthly_trends(combined)
    plot_top_seasonal(suii_calc)
    plot_population_scatter(combined)
    plot_geo_risk(geo_data, bio_risk)
    plot_youth_pies(combined)
    combined_sorted, anomalies_pct, anomalies_z = detect_anomalies(combined)
    print(f"Anomalies (pct change >100): {len(anomalies_pct)} | Anomalies (z-score>3): {len(anomalies_z)}")
    plot_anomaly_sample(combined_sorted, anomalies_z)
    correlation = education_correlation(suii_calc, oct_dec_data)
    if correlation is not None:
        print(f"Youth-seasonality correlation: {correlation:.3f}")
    comparison = rural_urban_comparison(combined, census)
    if comparison is not None:
        print("Rural vs Urban Comparison:\n", comparison)
    migration_bar(combined)
    recommendations(suii_calc, bio_risk, oct_dec_data, anomalies_z)
    print("Analysis complete")

```

```
if __name__ == "__main__":  
    main()
```